-- Date : 17-04-2026

-- SQl - Structured Query Language

-- SEQUEL - Structured English query Language

-- Database - A place where user can sore and retrive data

-- advantages --

-- sql has well defined standards

-- sql is easy to learn

-- in sql we can create multiple

-- sql querie are portable

-- no case sensitive

-- It is an interactive language

```
create database batch78;
use batch78;

set sql_safe_updates=0; -- It is used for uncheck safe update method wher 1 means check

-- Creating new table

create table student(id int,name varchar(20),age int);

-- select to see op as table
select * from student;
select name,age from student;

-- insert values
```

```
insert into student values(1,"Itachi",21);

insert into student (name,age) values("Shisui",24);


-- Error cuz  all column must have to put values


insert into student values(2,"sasuke");


--
_____
_____--


--                          Date : 20-04-2026


create table employee(id int, name varchar(20), dept varchar(20),salary int);

select * from employee;

insert into employee values(1,"Luffy","captain",500000);


insert into employee values(2,"Zoro","vice captain",300000);

insert into employee values(3,"Sanji","cook",300000);


insert into employee
values(5,"chopper","doctor",500),(4,"nami","navigator",150000),(6,"robin","archeologist",200000),(
7,"zoro","multiverse walker",300000);



-- drop

-- Deleting the whole structure of the table

-- not able to rollback


drop table employee;
```

-- truncate

-- structure remains but delete all the datas

-- also not able to rollback

truncate table employee;

-- delete

-- structure remains but delete all the datas

-- able to rollback

-- able to delete one row only

delete from employee;

-- commands

-- ddl

-- dml

-- dql

-- dcl

-- tcl

-- ddl - data definition language --

-- deals with database schemas description of how the

-- data should reside in db

-- create,alter,drop,truncate

-- dml data manipulation language --

-- deals with data manipulation

-- update,delete,insert

-- dql - data query language --

-- deals with retriving data from the tables

-- select

-- dcl - data control language --

-- deals with control of the database authorization

-- it includes cmd such as grant mostly concerned with

-- permisions and other control of the db system

-- grant revoke

-- tcl - transaction control language --

-- deals with a transaction with db

-- commit,rollback,savepoint

-- where

-- -- if we need only certain data from the tables

-- where clause acts as a filtering mechanism

```
select * from employee where  salary <= 300000;

select name from employee where salary >200000;

select * from employee where name ="sanji";
```

```
select * from employee where name ="brook";  -- no error -- o row(s) returned
```

```
-- And - OR - NOT

select * from employee where  name="zoro" and salary >= 300000;

select * from employee where  name="zoro" or salary >= 300000;

select * from employee where not name="zoro";

-- between operator

select * from employee where salary between 300000 and 600000;

select * from employee where not salary between 300000 and 600000;

select * from employee where salary between 300000 and 600000;

select * from employee where name between "luffy" and "sanji"; -- selext between thename by alphabatic order

select * from employee where not name between "luffy" and "sanji";

select * from employee where salary between 30000 and 60000;

-- in operator

select * from employee where name="luffy" or name="zoro" or name="sanji";  -- it is not easy to always for larger datasets

select * from employee where name in  ("luffy","zoro","sanji");

select * from employee where name not in  ("luffy","zoro","sanji");

-- order by

select * from employee order by name;
```

```
select * from employee order by name asc;

select * from employee order by name desc;

select * from employee order by salary;

select * from employee order by salary desc;

select * from employee order by salary desc,name;

select * from employee where salary between 200000 and 300000 order by salary,name;


--
_____
_____--

--                          Date : 21-04-2026

-- like operator -- exact under needed to match the numr of  values


-- __ single char



select * from employee where name like "z_ro";

select * from employee where name like "_a___";

select * from employee where name like "____";

select * from employee where name like "sanji_";

select * from employee where name like "_";




-- %  multiple char - here contrast 0 or more values to matching


select * from employee where name like "l%";

select * from employee where name like "l%y";

select * from employee where name like "%i";

select * from employee where dept like "%";

select * from employee where dept like "%o%";
```

-- Alias -- just for to see that time only

```
select id as rankk,name,dept as title,salary from employee;
select id as di from employee;
```

-- limit -- x,x == (offset , limit)

```
select * from employee order by salary desc limit 5;
select * from employee order by salary desc limit 1,1;
select * from employee order by salary desc limit 1,2;
select * from employee order by salary desc limit 2,1;
```

-- offset -- (limit x offset x)

```
select * from employee order by salary desc limit 5 offset 3;
select * from employee order by salary desc limit 1 offset 1;
select * from employee order by salary desc limit 1 offset 2;
select * from employee order by salary desc limit 2 offset 1;
```

-- distinct -- if a name exists 2 times in table it will not appear

```
select name,salary from employee;
select salary from employee;
```

```
select distinct name from employee; -- here zoro one time only
select distinct name,salary from employee;
```

```
select distinct salary,name from employee;  -- here sanji and  2nd zoro not appear in table
```

```
-- delete - is also used for removing a single row
set sql_safe_updates=0;
delete from employee where id =7;

select * from employee;

-- built_in functions

-- string  methods
-- numeric methods

-- date time methods
-- aggregate methods



-- string  methods

-- upper(x)

select "zoro";
select upper("zoro") as name;

select * from employee;
select *, upper(name) from employee;

select id,name,upper(dept),salary from employee;

-- lower(x)

select lower("Aizen") as name;
select *,lower(name) from employee;
```

```
select id,lower(name),dept,salary from employee;

-- length(x)

select length("Light Yagami");

select *, length(name) as len from employee;



--  instr(xxx,"x")

select instr("vegeta","e");
select name,instr(name,"ro") from employee;

-- substr("xxxxxxx",1,4)

select substr("vegeta",1,4);
select name,substr(name,1,4) from employee;



-- concat("xxxx",x)alter

select concat ("yo ","Ace");
select *,concat("yo ",name) from employee;



--
_____
_____--
--                          Date : 22-04-2026

-- trim
```

```
select "       Luffy.          ";
select trim("       Luffy.          ");

select length("       Luffy.          ");
select length(trim("       Luffy.          "));

-- Numeric function

-- abs

select abs(5);
select abs(-5);

-- mod()

select mod(10,2);
select mod(10,3);
select name,salary,mod(salary,7) from employee;

-- greatest by alphabets like z is the gteatest

select greatest ("luffy","zoro","sanji");

select max(name) from employee;
select max(salary) from employee;
select * from employee where name=greatest(name); -- its not working it can compare to all the rows

-- least()
```

```
select least ("luffy","zoro","sanji");

select least(10,20,30,40,50);
select min(salary) from employee;
select min(name) from employee;

-- pow() or power()

select pow(2,3);
select power(2,3);
select name,salary,pow(salary,1/10) from employee;
select name,salary,power(salary,1/10) from employee;

-- truncate - for rounding the value of point

select truncate(3.149874268585,2); -- it will show only upto the 2 decimals
select truncate(7456.149874268585,-3); -- it will replace 0 with befre point values
select truncate(7456.149874268585,-2);
select truncate(7456.149874268585,-1);

-- sqrt()

select sqrt(16);
select name,salary,sqrt(salary) from employee;

--
--
_____
_____--
--                          Date : 23-04-2026
-- election -- holiday
```

```
--
_____--
--                      Date : 24-04-2026
-- holiday
--
_____--
--                      Date : 25-04-2026
-- saturday -- leave
--
_____--
--                      Date : 26-04-2026
-- sunday -- holiday
--
_____--
--                      Date : 27-04-2026
-- assessment sql for saturday 6 questions


-- date()
--
select now();
select sysdate();
--
select year("2026-04-27");
select month("2026-04-27");
select day("2026-04-27");


-- year


select yearweek("2026-04-27");
```

```
-- month

select monthname("2026-04-27");

-- day

select dayname("2026-04-27");
select dayofmonth("2026-04-27");
select dayofweek("2026-04-27");
select dayofyear("2026-04-27");

-- week

select week("2026-04-27");
select weekday("2026-04-27");
select weekofyear("2026-04-27");


-- date

select datediff("2026-04-21","2026-04-27");
select datediff("2026-04-27","2026-05-7");

-- current

select current_date();
select current_time();
select current_timestamp();
select current_user();

-- aggregate functions
```

```
-- count
select count(salary) from employee;
-- sum
select sum(salary) from employee;
-- min
select min(salary) from employee;
-- max
select max(salary) from employee;
-- avg
select avg(salary) from employee;


-- batch 78 table

create table batch78(id int,name varchar(20),age int,joining_date date,dept varchar(20),salary int);
select * from batch78;

insert into batch78 values(1,"Anu",30,"2026-02-02","HR",40000),
(2,"Ruby",22,"2026-01-04","Admin",35000),
(3,"Karthik",29,"2026-01-07","Manager",85000),
(4,"Vicky",29,"2026-01-17","Dev",30000),
(5,"Eliyas",27,"2023-01-22","CEO",300000),
(6,"Dhanush",20,"2026-02-22","Tester",25000),
(7,"Guhan",27,"2026-03-12","HR",35000),
(8,"Akash",23,"2026-03-24","Dev",33000),
(9,"Hasan",21,"2026-04-03","Dev",35000),
(10,"Abimanyu",24,"2026-04-15","Tester",43000),
(11,"Mukesh",46,"2024-04-18","Dev",80000),
(12,"Thambi",25,"2024-03-26","Lead",60000);
```

```
insert into batch78 values(13,"Mukilan",44,"2025-08-20","Tester",30000);

select * from batch78;

insert into batch78(id,name,age,joining_date) values(14,"Seeman",51,"2021-07-14");

select * from batch78;



-- date substitute()

-- select date_sub("2024-04-18") from batch78;
select date_sub("2024-04-18",interval 3 year) from batch78;
select *,date_sub("2024-04-18",interval 3 year) from batch78 where id=7;
-- arithmatic operators

select name,salary,(salary+5000) incr from batch78; -- inctr is just a name
select name,salary,(salary-5000) decr from batch78;
select name,salary,(salary*2) incr from batch78;
select name,salary,(salary/2) decr from batch78;
select name,salary,(salary%2) remainder from batch78;
select name,salary,truncate((salary/2),0) decr from batch78;

drop table student;

-- student table

create table student select id,name, age from batch78;
select * from student;
```

--
_____
_____--

-- Date : 28-04-2026

```
insert into student values(15,null,40);
select * from student;

insert into student (id,name) values(16,"hari");
select * from student;

select * from student where name is null;
select * from student where name is not null;

select * from student where not name is null;
select * from student where not name is not null;

select * from student where name is null and age is null;
select * from student where name is null or age is null;

-- update

update student set name="kavi" where id=15;
select * from student;

create table stud1 select * from student;
select * from stud1;

update stud1 set name="muki"; -- it will change all name into muki



-- alter
```

```
desc stud1;
describe stud1;

-- add column
alter table stud1 add column batch varchar(10);
update stud1 set batch = "78";

-- modify

alter table stud1 modify column batch int;
desc stud1;
-- rename


alter table stud1 rename column `class name` to batch;
alter table stud1 rename column batch to `class name`;

--  drop column

alter table stud1 drop column `class name` ;



--
_____
_____--
--                          Date : 29-04-2026
-- leave
--
_____
_____--
--                          Date : 30-04-2026
```

-- group by

```
select dept,count(id) from batch78 group by dept;
select dept,min(salary) from batch78 group by dept;
select dept,max(salary) from batch78 group by dept;

select dept,sum(salary) from batch78 group by dept;



select dept,avg(salary) from batch78 group by dept;
select dept,sum(salary)/count(id) from batch78 group by dept;
select dept,truncate(sum(salary)/count(id),0) from batch78 group by dept;

select dept,name,salary from batch78 where (dept,salary) in (select dept,max(salary) from batch78
group by dept);
```

-- where -- having

-- filter from whole table filter from group by

-- before group by after group by

-- having

```
select dept,count(id) from batch78 group by dept having count(id);

select dept,count(id) from batch78 group by dept having count(id)>1;



select dept,count(id) from batch78 where salary>35000 group by dept;
```

```
select name,dept from batch78 where salary> 35000;

select dept,count(id) from batch78 where salary>=35000 group by dept having count(id)>1;

select name,dept,salary from batch78 where (dept) in (select dept from batch78 where
salary>=35000 group by dept having count(id)>1) and salary>=35000;

-- summa AI

select name,dept,max(salary) from batch78 group by name,dept; -- If you want to see the max salary
for every unique combination of name and department
                                        -- (though this usually just gives you everyone's individual salary):

--
_____
_____--
--                          Date : 01-04-2026

        -- set operators

create table A (id int,name varchar(20));

insert into A values(1,"itachi"),(2,"obito"),(3,"deidara"),(4,"kisamae");
select * from A;
create table B (id int,name varchar(20));

insert into B values(1,"sasuke"),(2,"obito"),(3,"shisui"),(4,"madara");
select * from B;

-- union -- dont allows duplicate name but allows ids

select * from A union select * from B;
```

-- union all -- allow duplicate

select * from A union all select * from B;

-- intersection

select * from A intersect select * from B;

-- temporary table -- if we exit workbench we have to run create table

-- it does not store in database batch78

create temporary table temp(id int,name varchar(20));

insert into temp values(1,"sasuke"),(2,"obito"),(3,"shisui"),(4,"madara");

select * from temp;

-- if not exits -- if a table not exists ,It will create new table.If a tabble exists, it will give warning not error.

create table if not exists A(id int,name varchar(20));

select * from A; -- here it also runs cuz it is not error

--
_____
_____--

-- Date : 02-04-2026

-- satuday -- assessment and mock

-- walk 1.8 kms walk - ss bucket biriyani parcel -- and Ate at footbath near ground

-- triples anbu-aakash-mukesh kodambakkam to perambur N4 Turf

-- play cricket 6:00 pm to 8:30pm full fun at N4 Turf perambur

-- sree balaji Ice cream -- mixed of vennilaa & pistha

-- triples anbu-mukilan-mukesh kodambakkam to perambur N4 Turf to kodambakkam

-- Happy night

--
_____
_____--

-- Date : 03-05-2026

-- sunday holiday

--
_____
_____--

-- Date : 04-05-2026

-- Monday -- Election result --

-- - is not available in my sql but available oracle

-- except is aavailable

select * from A except select * from B;

select * from B except select * from A;

```
-- joins



create table empl(id int,name varchar(20),dept int);

insert into empl values(1,"seeman",10),(2,"thambi",20),(3,"vicky",40);

select * from empl;

create table dept(dept int,name varchar(10));

insert into dept values (10,"ntk"),(20,"tvk"),(30,"admk");

-- inner join

select e.name ,d.name from empl e inner join dept d where e.dept = d.dept;

-- join also same

select e.name ,d.name from empl e join dept d where e.dept = d.dept;

-- left join means full left taable + matching data from right table

select e.name,d.name from empl e left join dept d on e.dept=d.dept;

-- right join

select e.name,d.name from empl e right join dept d on e.dept=d.dept;
```

-- fuul join is also not available in mysql but available in oracle

select name from empl union select name from dept;

select e.name,d.name from empl e left join dept d on e.dept=d.dept
union
select e.name,d.name from empl e right join dept d on e.dept=d.dept;

-- cross join -- cartesian product -- max no of variations -- like here 3x3 = 9

select e.name,d.name from empl e cross join dept d ;

--
_____
_____--
-- Date : 05-05-2026

-- constraints

-- Not Null -- does't allow null

create table C(id int not null,name varchar(20));
delete from C;
select * from C;
insert into C values(1,"ichigo");
insert into C (name)values("sado"); -- need id value also
insert into C values(null,"ishida");
insert into C (id)values(2);

-- unique - allows null values but does't allows duplicate

```
create table uniq(id int unique,name varchar(20));
drop table uniq;
select * from uniq;

insert into uniq values(1,"ace");
insert into uniq (name)values("sabo");
insert into uniq values(Null,"luffy");
insert into uniq values(1,"joyboy"); -- error cuz doesn't allows duplicate id - 1

-- primary key
-- neither allows duplicate values not null vaalues
-- if a primary key id is delete, by manually we can use 3, - by  auto increament -- skip 3 and goes to 4

-- null null row will automatically appear and it allows to give data in null shells for temporary

create table prim(id int primary key auto_increment,name varchar(20));
drop table prim;
select * from prim;

desc uniq;
desc prim;

insert into prim values(1,"alpha");
insert into prim values(2,"beta");
insert into prim values(3,"delta");

insert into prim values(null,"delta");
insert into prim values(1,"beta");
insert into prim values(null,null);
```

-- delte and inserting another time

```
delete from prim where id=3;
insert into prim (name)values("delta"); -- it will skip 3 and goes for 4
```

-- Auto increment

```
create table auto(id int primary key auto_increment,name varchar(20));
select * from auto;

insert into auto (name) values ("naruto"); -- it will auto matically id no with icrement one by one
insert into auto (name) values ("sasuke");
```

-- default Key

```
create table def (id int primary key auto_increment,name varchar(20),place varchar(20) default
"chennai",dept_id int not null);
drop table def;

select * from def;

insert into def (name,place)values("itachi","japan");
insert into def (name)values("lemon");
```

-- check

```
create table checkk(id int primary key auto_increment,name varchar(20),age int check(age>18));
select * from checkk;
```

```
insert into checkk (name,age) values ("luffy",19);

insert into checkk (name,age) values ("leo",18);

insert into checkk (name) values ("zoro");


create table cons(id int primary key auto_increment,name varchar(20) unique,age int default 19
check(age>17),place varchar(20) default "chennai");

select * from cons;

drop table cons;

insert into cons (name,age,place)values("ace",23,"windmill");

insert into cons (name) values("sabo");




--
_____
_____--

--                          Date : 06-05-2026


-- foriegn key


create table driver(dri_id int primary key,dri_name varchar(20));


select * from driver;


insert into driver values(101,"Asta"),(102,"yuno");


create table ride(ride_id int primary key,driver_id int,pick_up varchar(20),dropp varchar(20),foreign
key (driver_id) references driver(dri_id));


insert into ride values (301,101,"chn","tvl"),(302,102,"cbe","mdu");

insert into ride values(303,103,"mdu","tir");
```

```
select * from ride;

drop table driver; -- can't able to delete cuz it refernces used in ride table

delete from driver where dri_id = 101;

insert into driver values(110,"luck");

delete from driver where dri_id=110; -- can able to delete cuz it has no ride id refrences

select * from driver;

-- on delete cascade

create table driver1(dri_id int primary key,dri_name varchar(20));

select * from driver1;

insert into driver1 values(101,"Asta"),(102,"yuno");

create table ride1(ride_id int primary key,driver_id int,pick_up varchar(20),dropp varchar(20),foreign key (driver_id) references driver1(dri_id) on delete cascade);

select * from ride1;

drop table ride1 ;

insert into ride1 values (301,101,"chn","tvl"),(302,102,"cbe","mdu");

delete from driver1 where dri_id = 101;

select * from driver1;

select * from ride1;

-- on delete setnull

create table driver2(dri_id int primary key,dri_name varchar(20));
```

```
select * from driver2;


insert into driver2 values(101,"Asta"),(102,"yuno");


create table ride2(ride_id int primary key,driver_id int,pick_up varchar(20),dropp varchar(20),foreign key (driver_id) references driver2(dri_id) on delete set null);

select * from ride2;


drop table ride2 ;

insert into ride2 values (301,101,"chn","tvl"),(302,102,"cbe","mdu");


delete from driver2 where dri_id = 101;

select * from driver2;

select * from ride2;



create table s(id int primary key auto_increment,name varchar(20));

drop table s;

select * from s;

insert into s values(101,"ace");

insert into s(name) values("sabo");


create table user(id int primary key auto_increment,name varchar(10)) auto_increment=1000;


insert into user(name) values("mukesh");

insert into user(name) values("muke");

select * from user;

--
_____
_____--

--                          Date : 07-06-2026
```

-- on update cascade

```
create table driver3(dri_id int primary key,dri_name varchar(20));

select * from driver3;

insert into driver3 values(101,"Asta"),(102,"yuno");

create table ride3(ride_id int primary key,driver_id int,pick_up varchar(20),dropp varchar(20),foreign key (driver_id) references driver3(dri_id) on update cascade);

select * from ride3;

drop table ride3 ;

insert into ride3 values (301,101,"chn","tvl"),(302,102,"cbe","mdu");

update driver3 set dri_id=103 where dri_name="Asta" ;
```

-- on update set null

```
create table driver4(dri_id int primary key,dri_name varchar(20));

select * from driver4;

insert into driver4 values(101,"Asta"),(102,"yuno");

create table ride4(ride_id int primary key,driver_id int,pick_up varchar(20),dropp varchar(20),foreign key (driver_id) references driver4(dri_id) on update set null);

select * from ride4;

drop table ride4 ;

insert into ride4 values (301,101,"chn","tvl"),(302,102,"cbe","mdu");

update driver4 set dri_id=103 where dri_name="Asta" ;
```

```
-- TCL

-- rollback - ctrl + z
-- commit   - ctrl + s -- sving the rollback values
-- savepoint -

-- roolback and commit for insert

start transaction;
select * from student;
insert into student values(17,"prasana",21);
select * from student;
rollback;
commit;

--  rollback and commit for update

begin;
insert into student values(17,"surya",45);
select * from student;
rollback;

-- rollback and commit for update

start transaction;
select * from student;
update student set age=50 where name="surya";
select * from student;
rollback;
select * from student;
commit;
```

```
-- rollback and commit for delete

start transaction;
select * from student;
delete from student where age=45;
commit;

-- rollback for truncate

start transaction;

create table trunc (select * from employee);
select * from trunc;
drop table trunc;

rollback; -- does'nt working

-- rollback for drop

start transaction;
create table dropp (select * from employee);
select * from dropp;
drop table dropp;

rollback; -- also doesn't working
select * from dropp; -- no table

-- savepoint

-- here we can't rollback to before update
```

```
select * from student;
insert into student values(18,"mukesh",22);
select * from student;
update student set name="Anbu" where id=12;
select * from student;

commit;

--
start transaction;
select * from student;
delete from student where id=18;
savepoint beforee;

insert into student values(18,"prasanna",21);
select * from student;
savepoint afterr;

update student set name="mahi" where name="mukilan";
rollback to beforee;
select * from student;

commit;

--
_____
_____--
--                          Date : 08-05-2026

-- caase when then - else end
```

```
select
case
        when 10>5 then "true"
        else "false"
end;

select case when 10<5 then "true" else "false" end;

select name,age,
case
   when age>25 then "above 25"
   else "below 25"
end as eligible from student;

select * from batch78;

select name,age,
case
    when age=25 then "equal 25"
    when age>25 then "above 25"
    else "below 25"
end as eligible from student;

select name,age,joining_date,year(joining_date) from batch78;

-- store advantage

select * from employee;

-- calling like function
delimiter $$
```

```
create procedure calling()
begin
select * from employee;
end $$
delimiter ;
drop procedure calling;
call calling();

-- count

delimiter //
create procedure counting()
begin
    declare a int;
    select count(id) into a from student;
    select a;
end //

delimiter ;
call counting();
drop procedure counting;

-- parameter

-- in
-- out
-- inout

delimiter ^^

create procedure func()
```

```
begin
    select * from batch78 where dept="dev";
end ^^
delimiter ;
call func();

-- in

delimiter @@

create procedure phil(in jdesc varchar(20))
begin
    select * from batch78 where dept=jdesc;
end @@
delimiter ;
call phil("dev");

delimiter //
create procedure phil (in a int)
begin
    select * from student where dept = jdesc;
end //
delimiter ;

delimiter ..
create procedure alan(in a int)
begin
    select * from student where age>a;
end ..
delimiter ;
call alan(25);
```

call alan("ace") -- error cuz it not int

```
delimiter //
create procedure billa(in a int)
begin
    declare total int;
    select count(id) into total from student where age>a;
    select total;
end //

delimiter ;

call  billa(25);
call billa(100);
```

--
_____
_____--

-- saturday (assessment and mock) Date : 09-05-2026

--
_____
_____--

-- sunday (holiday) Date : 10-05-2026

--
_____
_____--

-- Date : 11-05-2026

```
-- out

delimiter //
create procedure outt(out total int)
begin
    select count(id) into total from batch78 where dept="Dev" ;
end //
delimiter ;

drop procedure outt;
call outt (@output);

select @output;
select * from batch78;
-- ----------------------------------------
-- in out

delimiter //
create procedure anbu(in a varchar(3),out total int)
begin
    select count(id) into total from batch78 where dept=a;
end //
delimiter ;

call anbu("dev",@total); -- here accepts only 3 varchar if more it will error

select @total;

-- set the total pre value and if we run call total will change
set @total = 10;
```

```
select @total;

call anbu ("hr",@total); -- here accepts only 3 varchar if more it will error

select @total;

-- ----------------------------

-- inout

delimiter //

create procedure inoutt (inout total int)

begin
        set total=total+5;
end //
delimiter ;
drop procedure inoutt;
set @count = 10;
call inoutt(@count);
select @count;

delimiter //
create procedure pokkie (inout total int,in a int)
begin
    set total=total+a;
end //
delimiter ;
drop procedure pokkie;
```

```
set @pokk=10;
call pokkie(@pokk,5);
select @pokk;



--


delimiter //
create procedure iff (in a int)
begin
    if a=1 or a=2 then
    select "hi";
    else
    select "bye";
    end if ;
end //
delimiter ;
call iff(2);
call iff(5);

delimiter //
create procedure casee (in a int)
begin
  case
     when a=1 or a=2
     then select "hi";
     else
     select "bye";
            end case;

end //
```

```
delimiter ;

call casee(2);
call casee(5);

 delimiter //
 create procedure cm(in a int)
 begin
     case a
     when 1
     then select "hi";
     when 2
     then select "hello";
     else
     select "bye";
     end case;
end //

delimiter ;
drop procedure cm;

call cm(1);
call cm(2);
call cm(6);



--
_____
_____--
--                          Date : 12-05-2026

-- loop
```

```
-- looping through multiple op page for each element in loop


delimiter ??
create procedure loopp()
begin
     declare n int;
     set n=1;
     looplable:loop
       if n=10 then
                          leave looplable;
               end if;
       select n;
       set n=n+1;
     end loop;
end ??
delimiter ;


call loopp();
-- looping through a concat str in  single a op page for each element in loop


delimiter //
create procedure looping()
begin
    declare n int;
    declare s varchar(30);


    set n=1;
    set s="";


    looplabel : loop
      if n=10 then
```

```
        leave looplabel;
                    end if;
      set s=concat(s," ",n);
      set n=n+1;
    end loop;
          select s;

end //
delimiter ;

call looping();

-- while loop

delimiter ..
create procedure whilee()
begin
    declare n int;
    declare s varchar(30);

    set n=1;
    set s="";

    while n<10 do
        set s=concat(s," ",n);
        set n=n+1;
    end while;
    select s;

end ..
delimiter ;
```

```
call whilee();
drop procedure whilee;

-- repeat until

delimiter ]]
create procedure repeatt()
begin
    declare n int;
    declare s varchar(30);

    set n=1;
    set s="";

    repeat
                    set s=concat(s," ",n);
                    set n=n+1;
            until n>10
    end repeat;
    select s;
end ]]
delimiter ;

call repeatt();
drop procedure repeatt;

-- _____

-- function

-- deterministic -- able to predict op
```

```
-- Non-deterministic -- Not able to predict op

--

delimiter ;;
create function func(age int)
returns varchar(50)
deterministic
begin
    case when age < 25
                    then return "Below 25";
                    when age = 25
        then return "Equal to 25";
        else return "Above 25";
            end case;
end ;;
delimiter ;

drop function func;
select age,func(age) from student;

delimiter ;;
create function funcif(age int)
returns varchar(50)
deterministic
begin
    if  age < 25
                    then return "Below 25";

            else return "Above 25";
            end if;
```

```
end ;;
delimiter ;

drop function funcif;

select func(27) as age;
select age,funcif(age) from student;

-- function and loop

delimiter //
create function lop(a int)
returns varchar(30)
deterministic

begin
    declare s varchar(30);
    set s="";
    looplable : loop
    if a>10 then
      leave looplable;
            end if;

    set s=concat(s," ",a);
    set a=a+1;
    end loop;
    return s;
end //
delimiter ;
```

```
select lop(1);

-- function and while do

delimiter //
create function whi(a int)
returns varchar(30)
deterministic

begin
    declare s varchar(30);
    set s="";
    while a<10 do
       set s=concat(s," ",a);
       set a=a+1;
    end while;
    return s;
end //
delimiter ;

select whi(1);
drop function whi;



--
_____
_____--
--                          Date : 13-05-2026

-- trigger
-- before insert
```

```
create table passenger(id int primary key auto_increment,name varchar(20),amount int);
insert into passenger (name,amount)values("mukilan",2000),("thambi",3000),("anbu",4000);
select * from passenger;

create trigger beforee
before insert
on passenger
for each row
set new.amount=new.amount+2000;
insert into passenger (name,amount)values("seeman",0);
select * from passenger;

-- after insert

create table manavar(id int primary key auto_increment,name varchar(30));

create table joining(id int primary key auto_increment,name varchar(20),audit datetime);

create trigger joining
after insert
on manavar
for each row
insert into joining(name,audit)values(new.name,now());

insert into manavar(name)values("anbu");
insert into manavar(name)values("selvam");

select * from manavar;
select * from joining;
```

```
-- before update

delimiter !!
create trigger beforeupdate
before update
on employee
for each row
begin
if new.salary<50000
   then set new.salary=50000;
end if;
end !!
delimiter ;

update employee set salary=40000 where id=5;
select * from employee;
update employee set salary=65000 where id=4;
select * from employee;

-- after update

create table afterr(id int,name varchar(20),updatename varchar(20));

create trigger afterupdate
after update
on employee
for each row
insert into afterr values(old.id,old.name,new.name);

select * from employee;
```

```
update employee set name="pirate_hunter" where name="zoro";
select * from employee;
select * from afterr;
update employee set name ="black_leg" where name="sanji";
select * from employee;
select * from afterr;

-- before delete

create table bdel(id int,name varchar(20),msg varchar(20));
delimiter !!
create trigger beforedel
before delete
on employee
for each row
begin
   insert into bdel values(old.id,old.name,"data will be delete");
end!!
delimiter ;

select * from employee;

delete from employee where name="nami";

select * from employee;
select * from bdel;

-- after delete

create table adel(id int,name varchar(20),salary int);
```

```
create trigger afterdel
after delete
on employee
for each row
insert into adel values(old.id,old.name,old.salary);

delete from employee where id=7;
select * from employee;
select * from adel;
```

--
_____
_____--

-- Date : 14-05-2026

```
create table allinone (id int primary key auto_increment,name varchar(20),age int);
drop table allinone;
insert into allinone (name,age)values("all might","46");

create table afinsert (id int,name varchar(30),age int);

create trigger affinsrt
after insert
on allinone
for each row
insert into afinsert values(new.id,new.name,new.age);

insert into allinone (name,age) values("todoroki",45);
```

```
select * from allinone;
select * from afinsert;

-- update --

create table afupdate (id int,oldname varchar(30),newname varchar(20));
drop table afupdate;
create trigger affupdate
after update
on allinone
for each row
insert into afupdate values(old.id,old.name,new.name);
drop trigger affupdate;

update allinone set name = "Endeavor" where age=45;

select * from allinone;
select * from afupdate;

-- after delete

create table afdelete (id int,name varchar(20),age int);

create trigger affdelete
after delete
on allinone
for each row
insert into afdelete values(old.id,old.name,old.age);

drop trigger affdelete;
```

```
delete from allinone where id=1;

select * from allinone;
select * from afdelete;

-- all three operations continously

insert into allinone (name,age)values("deku",17);
insert into allinone (name,age)values ("eraser head",35);
update allinone set name="deku" where age=17;
delete from allinone where id=5;

select * from allinone;
select * from afinsert;
select * from afupdate;
select * from afdelete;

-- _____--

-- Aggregate
-- ranking
-- values

create table sales(id int,store varchar(20),amount int);
drop table sales;

insert into sales values(1,"A",100),(2,"B",500),(3,"C",300),(4,"A",3500),(5,"B",4600);

select * from sales;

-- sum (group by)
```

```
select store,sum(amount) from sales group by store;

select id,store,amount,sum(amount) over (partition by store) from sales;

select *,sum(amount) over (partition by store order by id) from sales;

select *,sum(amount) over (partition by store order by id) from sales order by id;

select *,sum(amount) over (partition by store) from sales order by id;

-- min (groupby)

 select id,store,amount,min(amount) over (partition by store) from sales;

select *,min(amount) over (partition by store order by id) from sales;

select *,min(amount) over (partition by store order by id) from sales order by id;

-- max (groupby)

 select id,store,amount,max(amount) over (partition by store) from sales;

select *,max(amount) over (partition by store order by id) from sales;

select *,max(amount) over (partition by store order by id) from sales order by id;

-- avg (groupby)

 select id,store,amount,avg(amount) over (partition by store) from sales;
```

```
select *,avg(amount) over (partition by store order by id) from sales;

select *,avg(amount) over (partition by store order by id) from sales order by id;

-- count (groupby)

 select id,store,amount,count(amount) over (partition by store) as countt from sales;

select *,count(amount) over (partition by store order by id) from sales;

select *,count(amount) over (partition by store order by id) from sales order by id;

-- Ranking
-- row_number

select *,row_number() over (partition by store) from sales;

select *,row_number() over (partition by store order by amount) from sales;

select *,row_number() over(partition by store order by amount desc) from sales;

select *,row_number() over (partition by store order by amount desc)  from sales order by id;

select *,row_number() over() from sales order by amount desc;

-- rank -- if two mem have equal score it will give equal rank and skip a rank
-- (ranking with gap)
create table boys(id int primary key auto_increment,name varchar(20),score int);

insert into boys(name,score) values("anbu",85),("vicky",99),("eliyas",80),("akash",99);
select *,rank() over ( order by score desc) ranking from boys;
```

```
-- dense_rank -- here  if two mem have equal score it will give different rank

-- (ranking without gap)

select *,dense_rank() over (order by score desc) from boys;




-- highest mark from evaery dept in a clg with total rank


create table clg (id int primary key auto_increment, name varchar(20),dept varchar(10),score int check (score<=100)) ;


insert into clg (name,dept,score)values
("luffy","cse",56),("chopper","cse",100),("zoro","mech",40),("sanji","mech",80),("nami","ece",85),("robin","ece",95),("franky","civ",70),("jinbe","civ",75);


SELECT * FROM clg;


select rank() over(order by score desc) rankk from clg;



select name,dept,score from clg where (dept,score) in (select dept,max(score) from clg group by dept) ;


select dept,name,salary from batch78 where (dept,salary) in (select dept,max(salary) from batch78 group by dept);



--


SELECT r.name, r.dept, r.score, r.rankk

FROM (

   SELECT name, dept, score,
```

```
        RANK() OVER (ORDER BY score DESC) AS rankk
    FROM clg
) r
WHERE (r.dept, r.score) IN (
    SELECT dept, MAX(score)
    FROM clg
    GROUP BY dept
);


--


WITH ranked_clg AS (
    SELECT name, dept, score,RANK() OVER (ORDER BY score DESC) AS rankk    -- Your 1st query logic
    FROM clg
)
SELECT name, dept, score, rankk
FROM ranked_clg
WHERE (dept, score) IN (SELECT dept, MAX(score)
    FROM clg GROUP BY dept                        -- Your 2nd query logic
    );


--
_____________________________________________________________________________________
_____________________________________________--
--                              Date : 15-05-2026


-- percent rank


select * ,percent_rank() over(order by score desc) ranking from boys;



-- ntile  -- like group by but it selects group member by its own order
```

```
select * from student;

select *,ntile(4) over() from student;
select *,ntile(4) over(order by age desc) from student;

select *,ntile(4) over(order by score) from boys;



-- values

-- lag() - previous data

create table emp1(id int,name varchar(20),salary int,yearr year);

insert into emp1 values
(1,"mukilan",4000,2024),(1,"mukilan",8000,"2025"),(1,"mukilan",12000,"2026"),
                (2,"anbu",30000,"2025"),(2,"anbu",40000,"2026"),
                (3,"vicky",100000,2023),(3,"vicky",300000,2026),
                (4,"eliyas",35000,2026),(4,"eliyas",40000,2026);
select * from emp1;

select id,name,yearr,salary,lag(salary) over (partition by name) as previous from emp1  ;

-- lag difference between current salary and previous salary:

select id,name,yearr,salary,lag(salary) over (partition by name) as previous, salary-lag(salary) over
(partition by name) as differnce from emp1  ;

-- lead() increment between of current and next salary:

select *,lead(salary) over (partition by name) as nextt from emp1;
```

```
select id,name,yearr,salary,lead(salary) over (partition by name) as nextt,(lead(salary) over (partition by name) - salary) as increment from emp1;


-- first_value()


select *,first_value(salary) over(partition by name) from emp1;


select *,first_value(salary) over(partition by name),salary-first_value(salary) over(partition by name) as total_diff from emp1;


-- last_value()


select *,last_value(salary) over(partition by name) from emp1;


select *,last_value(salary) over(partition by name),last_value(salary) over(partition by name)-salary as total_diff  from emp1;


-- order by yearr is in last value will be thaat years max salary


select *,last_value(salary) over(partition by name order by yearr) from emp1;


-- max salary by (name) each person -- unlike above


select *,last_value(salary) over(partition by name order by yearr rows between current row and unbounded following) from emp1;


-- nth_value()


select *,nth_value(salary,2) over(partition by name) from emp1;


select *,nth_value(salary,2) over(partition by id order by yearr desc rows between current row and unbounded following) from emp1;
```

```
-- exception handling

select * from boys;

insert into boys values(3,"karthik",25);

delimiter //

create procedure exception(in id int,in name varchar(20),in age int)
begin
    insert into boys values(id,name,age);
    select * from boys;
end //
delimiter ;

call exception(5,"mukesh",22);
call exception(4,"guhan",27);

delimiter //
create procedure excontinue(in id int,in name varchar(20),in age int)
begin
     declare continue handler for 1062
     begin
                    select concat("duplicate key",id,"entry");
            end ;
      insert into boys values(id,name,age);
      select  * from boys;
end //
delimiter ;
```

```
call excontinue(4,"kavi",22);


delimiter //
create procedure exexit(in id int,in name varchar(20),in age int)
begin
    declare continue handler for 1062
    begin
                    select concat("duplicate key",id,"entry");
            end ;
     insert into boys values(id,name,age);
     select  * from boys;
end //
delimiter ;




call exexit(4,"kavi",22);




--
_____
_____--
--                          Date : 16-05-2026
-- saturday  mock and sql assessment
--
_____
_____--
--                          Date : 17-05-2026
-- sunday -- holidy
```

```
--
_____
_____--
--                          Date : 18-05-2026

-- 1062 -- duplicate value
-- 1503 -- unknown column
-- 1406 -- too long for column

-- signal

select * from student;

desc boys;
select * from boys;

insert into boys(name,score) values ("karthidfghjklsdfghjtyuighjkl",93);

delimiter //

create procedure signall(
  in id int,
  in name varchar(20),
  in score int)

begin
    declare continue handler for 1062
    begin
       select concat("duplicate key",id,"entry");
    end;
    if length(name)<=4 then
    signal sqlstate "45000"
```

```
    set message_text="name is too short";
    end if;
    select * from boys;
    insert into boys values(id,name,score);
    select * from boys;

end //
delimiter ;

call signall(6,"karthi",87);
call signall(7,"karthi",87);
call signall(8,"ruby",85);
insert boys(name,score) values("anu",89);
select * from boys;

-- resignal

delimiter //
create procedure resignall(
   in id int,
   in name varchar(20),
   in score int
)

begin
    declare continue handler for 1062
    resignal set message_text="error";
    insert into boys values(id,name,score);
    select * from boys;
end //
delimiter ;
```

```
call resignall (4,"fghj",45);
select * from boys;

call resignall (10,"prasanna",45);

-- cursor
select * from batch78;
select * from boys;
create table duplicate_boys(id int,name varchar(20),score int);

delimiter //
create procedure cursorr()
begin
    declare z int default 0;
    declare id int;
    declare name varchar(20);
    declare score int;
    declare cur cursor for select * from boys;
    declare exit handler for not found set z=1;
    delete from duplicate_boys;
    open cur;
        looplabel:loop
                    if z=1 then leave looplabel;
      end if;
      fetch cur into id,name,score;
      insert into duplicate_boys values(id,name,score);
      end loop;
           close cur;
end //
delimiter ;
```

```
select * from duplicate_boys;
call cursorr();

create table dup_boys select * from boys;
select * from dup_boys;
drop table dup_boys;

-- perfect number

delimiter //
create function perfect_num (n int)
returns varchar(30)
deterministic

begin
    declare i int;
    declare count int;
    set i = 0;
    set count = 0;
    while i<n do
       if n%i = 0 then
                                set count=count+i;
                    end if;
       set i=i+1;
            end while;
    if n=count then
      return "perfect number";
            else
                    return "not a perfect number";
            end if;
```

```
end //
delimiter ;

select perfect_num(6);



delimiter //

create procedure perfect(in n int)
begin
    declare i int;
    declare count int;
    set i = 0;
    set count = 0;
    while i<n do
       if n%i = 0 then
                                set count=count+i;
                    end if;
       set i=i+1;
            end while;
    if n=count then
      select "perfect number";
            else
                    select "NOt a perfect number" ;
            end if;
end //
delimiter ;

call perfect(6);
call perfectt(7);
```

-- composite primary key

-- It means have more than one primary key -- with inter connected two columns

-- minimal key

-- primary key(student_id,course_id)

-- the key is minimal bcuz neither student_id nor course_id is alone

-- can uniquely identify a row removing either a column would lasty uniqueness

-- Natural key

-- ph:no,aadhar no,pan no are naturally unique key

-- surrogate key

-- where key is undefined in table,if we want we can use auto_increment

-- canditate key

-- all primary keys are candidate key

-- all cadidate keys are composite keys

-- cumpolsary that column only present then primaary key constraints followa that key aare candidate keys

-- super key

-- all primary kesy are super key

-- all candidate keys are super key

--
_____
_____--

-- Date : 19-05-2026

```
use batch78;
set sql_safe_updates=0;

-- sub query

-- single row sub query

select * from employee;

select max(salary) from employee;

select * from employee where salary=(
select max(salary) from employee);

-- Multi row sub query

-- in
-- any
-- all

select * from batch78;

select name,dept from batch78 where dept in (
select dept from batch78 where length(dept)=3);

select * from batch78 where dept in (
select dept from batch78 where dept like "___");

-- any atleast one value  highest value
```

-- > ANY: Returns true if greater than the minimum value in the set.

-- > ALL: Returns true if greater than the maximum value in the set

-- > greater than minimum one

-- all

```
select * from batch78;
select * from batch78 where dept="hr";
select * from batch78 where salary > any(
select salary from batch78 where dept="hr");

select * from batch78 where salary < any(
select salary from batch78 where dept="hr");
```

-- > max value from greater than

-- < min value from lessthan

-- any all

-- > min ⬆️ max ⬆️

-- < max ⬇️ min ⬇️

```
select * from batch78 where salary > all(
select salary from batch78 where dept="hr");

select * from batch78 where salary < all(
select salary from batch78 where dept="hr");

select * from employee;
```

```
select * from employee where salary > any(
select salary from employee where salary in(150000,500));


select * from employee where salary > all(
select salary from employee where salary in(150000,500));

select * from employee where salary < any(
select salary from employee where salary in(150000,500));

select * from employee where salary < all(
select salary from employee where salary in(150000,300000));


-- correlated sub query

-- inner query depended on the outer query

-- find the emplloyees who are recieve more than dept avg salary

select id,name,dept,salary from batch78 b
where salary > (
select avg (salary) from batch78 where dept=b.dept);


-- nested sub query

select id,name,dept,salary from batch78 where dept = (
select dept from batch78 where salary=(
select max(salary) from batch78)) ;
```

```
select id,name,dept,salary from batch78 where dept = (

select dept from batch78 where salary=(

select min(salary) from batch78)) ;


--


create table product (p_id int primary key auto_increment,name varchar(20),price int);

create table customer (c_id int primary key auto_increment,p_id int,name varchar(20),foreign key (p_id) references product(p_id));

create table orderr (o_id int primary key auto_increment,c_id int,foreign key (c_id) references customer(c_id));


drop table orderr;


insert into product values(1,"bat",500),(2,"ball",50),(3,"stump",300);

select * from product;

insert into customer values(101,1,"mukesh");

insert into customer values(102,2,"prasanna"),(103,1,"guhan");

select * from customer;


insert into orderr values (1001,101),(1002,102),(1003,103);

select * from orderr;


--- eg for nested sub query


select c_id from orderr where o_id=1001; -- op==101


select p_id from customer where c_id = 101; -- op == 1


select name from product where p_id = 1;


-- nested sub query
```

```
select * from product where p_id = (
select p_id from customer where c_id=(
select c_id from orderr where o_id=1001));



-- scalar sub query

select (select max(salary) from batch78) a;

create table customerr (id int, name varchar(20));
insert into customerr values (1,"mukilan"),(2,"mukesh"),(3,"akash"),(4,"anbu");

select * from customerr;
-- foreign key (cust_id) references id
create table orde (id int, cust_id int,amount int);
insert into orde values(101,1,500),(102,1,400),(103,2,100);
drop table orde;
select * from orde;

-- exists

-- find the customer who placed orders

select name from customerr c where exists (
select * from orde o where o.cust_id=c.id);

-- not exists opposite of exists
```

```
select name from customerr c where not exists (
select * from orde o where o.cust_id=c.id);

--
_____
_____--
--                          Date : 20-05-2026

use batch78;
set sql_safe_updates=0;

-- cte - common table expression -- can not uase in where clause

with abc as(select name,dept,salary from batch78 where dept="dev")
select * from abc where salary > 33000;

-- cluster index  - which  can be used in where uniue or primary key  -- only one can be added in a table
-- non cluster index - any number of can be added in a table
--

create index idx on employee(id);
drop index idx on employee;
select * from employee where id=5;
explain analyze select * from employee where id=7;

--      Advantages                  Disadvantages

--      faster in

-- cluster index
```

-- can't use in built in function

-- in small table use is waste cuz time diff is low

-- primary key have default index in it

-- ACID -- Atomicity , Consistency , Isolation , Durability

-- Atomicity (All or nothing)

-- if transaction means either full commit or full rollback

-- Consistency (valid rules)

-- maintain the database valid rules

-- Isolation (independent) (no interference)

-- do not iterfere when multibple transaction happenning -- eg flight ticket booking

-- Durability (permanent save)

-- Guarantees the transaction once commited are must be permanently saved even server crash

-- why ACID property

-- prevents partial updates, Data curruption, Concurrency issue.

-- Normalization

-- reduandancy, update anomaly, insert anomaly, delete anomaly

-- Types

-- 1 NF (normal form)

-- each column shold contains atomic values (single values) (multiple values are not allowed)

-- 2 NF (normal form)

-- must be in 1 NF , no paartial dependency

-- 3 NF (normal form)

-- must be in 2 NF, Have no transitive dependencies

-- remove indirect dependency

-- boyce-CODD NF - BCNF (normal form)

-- strict version of 3NF , Aso called 3.5 NF

-- every determinant shoud be candidate key

-- table has multiple overlapping candidate key

-- for every functional dependency and every deprndency must be super key

-- 4 NF

-- remove multiple values dependencies

-- 5 NF (rarely used very advanced)

-- remove joined dependency

-- Normalization NF Advantages

-- Less duplicates

-- better consistentcy

-- NF Disadavantages

-- more joins becomes more complex

-- reads slowly cuz too many tables

-- DeNormalization

-- Reads easily and analyzing also easy

-- 1 NF -- Multiple values are removed

-- 2 NF -- no partial dependency

-- 3 NF -- no transitive dependency

-- BCNF NF -- strict version of 3 NF

-- 4 NF -- multivalue dependency

-- 5 NF -- remove join

```
select dept,avg(salary) from batch78 group by dept;
```

--
_____
_____--

-- Date : 21-05-2026

-- sem - exam -- leave

-- grand

```
select * from mysql.user;

create user "Anbu"@"localhost" identified by "Anbu@1808";
select * from mysql.user;

show grants for "root"@"localhost";
show grants for "Anbu"@"localhost";
grant select on batch78.* to "Anbu"@"localhost";
select * from mysql.user;
show grants for "Anbu"@"localhost";
grant insert on batch78.* to "Anbu"@"localhost";
```

```
show grants for "Anbu"@"localhost";
grant update,delete on batch78.* to "Anbu"@"localhost";
show grants for "Anbu"@"localhost";

grant all privileges on batch78.* to "Anbu"@"localhost" with grant option;

create user "Selvam"@"localhost" identified by "Selvam@1808";
show grants for "Selvam"@"localhost";

-- revoke

revoke all privileges,grant option from "Anbu"@"localhost";
show grants for "Anbu"@"localhost";

grant select on batch78.* to "Anbu"@"localhost";
show grants for "Anbu"@"localhost";

revoke select on batch78.* from "Anbu"@"localhost";
show grants for "Anbu"@"localhost";

-- change the table name
alter table bat rename to batch78;
select * from batch78;

-- user name change

update mysql.user set user="Anbuuu" where user="Anbu";
select user,host from mysql.user;

-- password change
```

```
alter user  "Anbuuu"@"localhost" identified by "Anbu@18082002";


drop user "Selvam"@"localhost";
select user,host from mysql.user;



-- view


use batch78;
select * from batch78;


create view less25 as select * from batch78 where age>25;


--
_____
_____
```